

Lecture 5

บท: Recursion

วิชา: 819605 Discrete Mathematics

อาจารย์ผู้สอน: ปพนธีร์ แยมโชติ | วิศวกรรมคอมพิวเตอร์ | ภาคปลาย ปีการศึกษา 2568

เป้าหมายการเรียนรู้ของเอกสารประกอบนี้ (Week 5): เน้นที่การ solving มากกว่า proving

1. เข้าใจหลักการของแนวคิดการแก้ปัญหาด้วยวิธีการเวียนเกิด (recursion)
2. สามารถพิจารณาลักษณะของปัญหาที่สามารถแก้ด้วยการเวียนเกิดได้
3. และสามารถแก้ปัญหาดังกล่าวด้วยวิธีการเวียนเกิดได้
4. (optional) สามารถ implement การเวียนเกิดด้วยการเขียน code ได้ (สอนด้วย C (อาจจะ Python ขึ้นกับข้อสรุปในห้อง))

1 แนวคิดพื้นฐานของการเวียนเกิด

การเวียนเกิด (recursion) คือรูปแบบการแก้ปัญหา (หรือการคำนวณ) ปัญหาหนึ่ง ๆ ที่อาศัยผลลัพธ์จากการแก้ปัญหาเดียวกันแต่ input เล็กกว่านำมาประกอบขึ้นเป็นผลลัพธ์ของปัญหาที่กำลังแก้

ซึ่งในทางการเขียนโปรแกรมนั้น input ที่เล็กกว่ามักจะเป็นส่วนย่อยของ input ที่อยากแก้ปัญหา ตัวอย่างเช่น

- อยากแก้ปัญหาที่รับ array $A[1..n]$ เป็น input โดยใช้ผลลัพธ์จากโจทย์เดียวกันแต่แก้กับ $A[1..(n-1)]$ (กล่าวคือลองดูกับแค่ตัวแรกจนถึงตัวรองสุดท้ายก่อนว่าได้ผลลัพธ์เป็นอะไร แล้วค่อยนำคนสุดท้ายมาพิจารณาพร้อมผลลัพธ์ที่ได้มา)
- อยากคำนวณค่าของโจทย์หนึ่งที่รับ n เข้าไป โดยใช้ผลลัพธ์จากโจทย์เดียวกันที่คำนวณกับ $n-1$ และ $n-2$ มาแล้ว

การแก้ปัญหาทาง computational บนโลกนี้มี 2 แนวคิดหลัก

1. **Iterative programming:** ใช้ loop ที่ผู้พัฒนาตั้ง logic การแก้ปัญหาในแต่ละรอบไว้แบบชัดเจนแล้ว -- อ่านง่าย(สำหรับคนส่วนใหญ่) วิธีการแก้ปัญหาเหมือนพาจับมือทำ ภาษาโปรแกรมส่วนใหญ่ออกแบบมาเพื่อเขียนแบบนี้อยู่แล้ว แต่การได้วิธีการแก้บางโจทย์ค่อนข้างยากและไม่ตรงไปตรงมา และตรวจสอบความถูกต้องทางทฤษฎีได้ลำบาก
2. **Recursive programming:** ฟังก์ชันเรียกใช้ตัวเองเพื่อเอาคำตอบจากปัญหาที่เล็กกว่า -- บางคนบอกว่าอ่านยาก(แต่สำหรับผม code คลีนกว่า) วิธีการได้มาของผลลัพธ์เหมือนต้องอาศัยความรู้เก่าเลยไม่รู้ว่าจะได้มายังไง ภาษาแบบ functional programming ถูกพัฒนาเพื่อเขียนแบบนี้ (เช่น Haskell, Scala) แต่ภาษาที่ iterative ก็ทำได้ แต่ว่าการคิดวิธีการแก้โจทย์ค่อนข้างชัดเจนและเป็นธรรมชาติกับวิธีคิดมากกว่า และที่สำคัญ ตรวจสอบความถูกต้องของการทำงานของโปรแกรมด้วยการพิสูจน์ทางคณิตศาสตร์ได้ (อาจารย์คิดว่าวิธีคิดของมนุษย์ใกล้เคียงการแก้ปัญหแบบ recursive มากกว่า iterative เพราะปกติมนุษย์มักใช้ประสบการณ์ในการลองแก้ปัญหาที่ง่ายกว่าก่อนแล้วค่อยพัฒนาผลลัพธ์ปัญหาย่อยให้เป็นผลลัพธ์ของปัญหาที่ยากขึ้น)

Example 1. ขอเริ่มที่ตัวอย่างที่ยังไม่ใช้การแก้ที่ปัญหา แต่เป็นเรื่องของวิธีการคำนวณแบบเวียนเกิดเพื่อให้เข้าใจรูปแบบการคำนวณแบบเวียนเกิดก่อน ซึ่งคือลำดับที่รู้จักกันในชื่อ "ลำดับเลขคณิต" (arithmetic sequence) ที่นิยามแบบคำอธิบายภาษามนุษย์ว่า "คือลำดับที่ผลต่างของพจน์ใด ๆ และพจน์ก่อนหน้าจะมีค่าคงที่เสมอในทุก ๆ พจน์" ถ้ากำหนดสัญลักษณ์สำหรับพจน์ที่ n ให้เขียนแทนด้วย $a(n)$ และสมมติให้ค่าคงที่คือ d เราจะเขียนอธิบายข้อความดังกล่าวเป็นภาษาคณิตศาสตร์ได้เป็น:

$$\forall n \in \mathbb{N}, \underline{\hspace{10cm}}$$

ถ้าสมมติให้ $a(0) = 5$ และ ค่าคงที่ที่กล่าวถึงคือ $d = 3$ จงคำนวณหา $a(7)$

Example 2. อีกตัวอย่างการคำนวณคือ Fibonacci ซึ่งมีสมการนิยามคือ

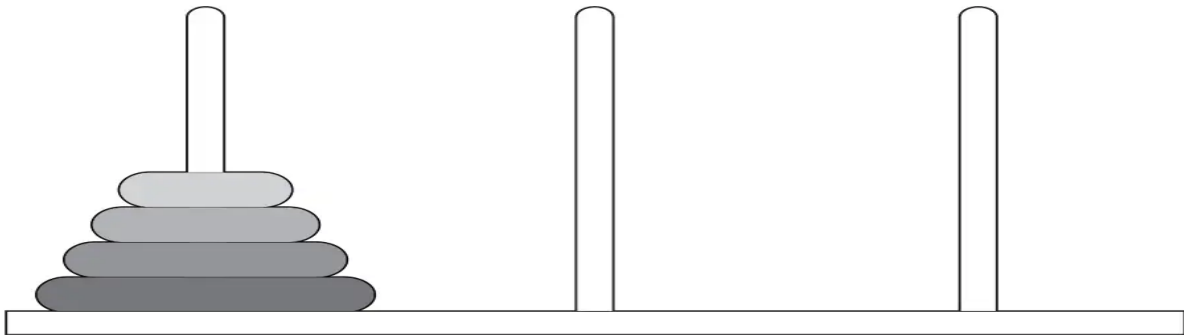
$$Fib(n) = Fib(n - 1) + Fib(n - 2)$$

โดยที่ $Fib(0) = 1$ และ $Fib(1) = 1$

จงคำนวณหา $Fib(7)$

จากตัวอย่างการคำนวณ 2 ข้อที่ผ่านมา นักศึกษาสังเกตเห็นอะไรที่เป็นส่วนสำคัญของการนิยามการคำนวณแบบเวียนเกิดบ้าง (ถามอีกแบบคือ ถ้าเอาเงื่อนไขอะไรออกจะทำให้การคำนวณเราพัง)

2 First Recursion: เกม Tower of Hanoi (เล่นอย่างไร + ต้องเล่นกี่ครั้ง)



ลองพิจารณาเกม *Tower of Hanoi* ที่มีเสา 3 ต้น (ให้เรียกชื่อว่า A, B, C) และมีแผ่นดิสก์ซ้อนกันอยู่บนเสาต้นหนึ่ง โดยมีกติกาว่า

- ย้ายได้ครั้งละ 1 แผ่นเท่านั้น
- ห้ามวางแผ่นใหญ่ทับแผ่นเล็ก (แผ่นเล็กกว่าอยู่ด้านบนของแผ่นใหญ่กว่าเสมอ)

ตอนนี้ให้สมมติสถานการณ์ดังนี้:

คุณกำลังอยากเล่น *Tower of Hanoi* ที่มี 4 แผ่นโดยนึกขึ้นได้ว่ามีเพื่อนคนหนึ่งที่มีความสามารถพิเศษว่า

ความสามารถพิเศษของเพื่อน:

สำหรับเกมที่มี 3 แผ่น และไม่ว่าจะเริ่มต้นด้วยการวางแผ่นทั้ง 3 ซ้อนกันอยู่บนเสาต้นไหนก็ตาม เพื่อนของคุณสามารถย้ายทั้ง 3 แผ่นไปยังเสาอีกต้นหนึ่ง (ที่คุณบอก) ได้ถูกต้องเสมอ โดยใช้เสาที่เหลือเป็นตัวช่วยตามกติกา

กล่าวคือ เพื่อนของคุณเป็น ``ผู้เชี่ยวชาญ *Tower of Hanoi* 3 แผ่น" ที่คุณสามารถสั่งได้ว่า

``ช่วยย้าย 3 แผ่นจากเสา X ไปเสา Y ให้หน่อย"

และเขาจะทำได้สำเร็จให้คุณเองโดยอัตโนมัติ

โจทย์: ตอนนี้คุณต้องการออกแบบวิธีเล่นเกม *Tower of Hanoi* แบบ 4 แผ่น โดยให้เพื่อนคนเก่งที่เล่นแบบ 3 แผ่นเป็นผู้ช่วยอยู่เบื้องหลัง เป้าหมายคือ

- เริ่มต้น: แผ่นทั้ง 4 ซ้อนกันอยู่บนเสาต้นหนึ่ง (เช่น เสา A)
- เป้าหมาย: ย้ายแผ่นทั้ง 4 ไปยังอีกเสาต้นหนึ่ง (เช่น เสา C)
- ใช้เพื่อนคนเก่ง 3 แผ่นเป็น ``ชายชุดดำ" ช่วยย้ายชุดของ 3 แผ่นเล็กตามที่คุณสั่ง (โดยที่คุณไม่จำเป็นต้องเข้าใจหรือรู้เห็นว่าเล่นอย่างไร รู้แค่คำสั่งเมื่อไหร่คุณจะได้เห็นกอง 3 แผ่นนั้นย้ายไปในที่ปรารถนา)

ให้ตอบคำถามต่อไปนี้

1. สมมติว่าเริ่มต้นมีแผ่นทั้ง 4 อยู่บนเสา A และคุณต้องการย้ายไปที่เสา C โดยใช้เสา B เป็นเสาดำช่วย
 - a) ในมุมมองของคุณ แผ่น **แผ่นล่างสุด** (แผ่นที่ใหญ่ที่สุด) ควรถูกย้ายจากเสา A ไปเสา C **เมื่อไร** ในลำดับการเล่น (ต้นเกม กลางเกม หรือท้ายเกม) และเพราะอะไร?
 - b) ไม่ว่าคุณจะกำหนดให้แผ่นใหญ่สุดอยู่ที่เสาใดก็ตาม ทำไมถึงไม่กระทบการเล่น 3 แผ่นที่เหลือของเพื่อนผู้ช่วย
 - c) ก่อนที่คุณจะย้ายแผ่นล่างสุดจาก A ไป C คุณจำเป็นต้องจัดการกับแผ่นเล็กอีก 3 แผ่นด้านบนอย่างไรบ้าง?
2. ตอนนี้ให้คุณ **เขียนแผนการเล่น 4 แผ่นแบบทีละขั้น** โดยใช้เพื่อนคนเก่ง 3 แผ่นเป็น “ชายชุดดำ” (ไม่ต้องสนใจรายละเอียดว่าเขาย้ายอย่างไร):

แผนการเล่น Tower of Hanoi 4 แผ่น (จาก A ไป C โดยใช้ B เป็นตัวช่วย):

จากแผนข้างบน ลองเขียนสรุปให้เป็น “ขั้นตอนวิธี” (เป็นภาษาคน) สำหรับการย้าย **4 แผ่น** ว่า

“ในการย้าย Tower of Hanoi 4 แผ่นจากเสาเริ่มต้นไปยังเสาปลายทาง เราจะ”

(1) _____

(2) _____

(3) _____

(4) _____

3. (Beyond specific case) จากไอเดียการเล่น 4 แผ่นข้างต้น ถ้าต้องการออกแบบวิธีเล่นสำหรับ n แผ่น ใด ๆ เราต้องการอะไรบ้าง และจะออกแบบขั้นตอนวิธีการเล่นอย่างไร

4. (Extended Question) ในกรณีของเกม n แผ่นที่นักศึกษา มีเพื่อนคนเก่งที่เล่นเกม $n - 1$ แผ่นได้เทพมาก ๆ คราวนี้นักศึกษาเกิดคำถามขึ้นมาเกี่ยวกับจำนวนตาที่ต้องใช้เล่นว่าถ้าอยากเล่นให้เทพที่สุดก็ต้องเล่นแบบที่ใช้จำนวนรอบการย้ายให้ต่ำที่สุด ถ้าสมมติว่าเพื่อนของนักศึกษาถูกฝึกให้เล่นเกม $n - 1$ แผ่นโดยใช้จำนวนรอบการย้ายแผ่นน้อยที่สุดแล้ว จงหาว่าสำหรับเกม n แผ่นนั้นจะต้องย้ายกี่รอบ

3 Second Recursion: หั่นแผ่นกระดาษได้มากที่สุดกี่ชิ้น

Example 3. สมมตินักศึกษาที่มีแผ่นกระดาษที่มีขอบเขตไม่สิ้นสุด (แผ่นระนาบ) ในโจทย์นี้ เราสนใจจำนวนชิ้นจากการลากเส้นตรงแบ่งกระดาษ โดยต้องการแบ่งให้ได้จำนวนชิ้นให้มากที่สุด ซึ่งชัดเจนกว่าถ้านักศึกษาลากเส้นตรง 1 เส้น จะแบ่งกระดาษได้ออกเป็น 2 ชิ้น และถ้ายากเส้นตรง 2 เส้นให้ตัดกัน จะแบ่งกระดาษออกเป็น 4 ชิ้น (และแบ่งได้มากที่สุดแล้วด้วย 2 เส้น) จงหาว่าถ้าเราจะตัดกระดาษด้วยเส้นตรง n เส้น (เมื่อ n เป็นจำนวนนับ) แล้วจำนวนชิ้นจากการแบ่งได้มากที่สุดจะเป็นกี่ชิ้น

4 Third Recursion: เดินขึ้นบันไดให้ใช้จำนวนก้าวที่น้อยที่สุด

Example 4. ในการเดินขึ้นบันได n ชั้น กำหนดให้เดินขึ้นได้ 2 แบบ คือ เดินขึ้นทีละ 1 ชั้น หรือทีละ 2 ชั้น จงเขียนสมการความสัมพันธ์เวียนเกิดแสดงจำนวนวิธีในการเดินขึ้นบันไดว่ามีกี่วิธีที่แตกต่างกันที่จะเดินไปชั้นที่ n

5 Fourth Recursion: จำนวนเซตย่อย

Example 5. จงหาจำนวนเซตย่อยทั้งหมดของเซตที่มีสมาชิก n ตัว โดยสมมติว่าเป็นเซต $A = \{1, 2, 3, \dots, n\}$